

Product Template Update Tracking — Architecture Design

***Problem, in one sentence:** keep an up-to-date, cheap-to-query picture of how far behind each engagement is from its template's latest version — without paying the 1-minute-per-engagement load cost — and present the accumulated changes in a form a non-technical auditor can act on.*

1. High-Level Architecture

The core move is decoupling *knowing* an engagement is behind from *loading* it. The engagement system already touches `template_id/version` at three moments we're allowed to hook: creation, apply, and decline. Those hooks populate a separate, cheap index — the index becomes the source of truth for “is this engagement behind,” and the slow engagement store is only ever touched during those same three moments, which already happen for other reasons.

Components

- **EngagementTemplateState** (DynamoDB): `engagement_id` (PK), `firm_id`, `template_id`, `current_version`, `pending_update`, `latest_available_version`, `updated_at`. Written exclusively by hooks on the engagement system — never by loading or polling engagements.
- **TemplateVersions registry**: `template_id + version` (PK), `published_at`, `diff_json` (raw diff — Level 3), `diff_structured` (deterministic old-vs-new field records — Level 2), `diff_summary` (LLM prose generated from Level 2 — Level 1), `summary_status`. One row per version, shared across every firm using that template.
- **Event bus** (e.g. EventBridge): decouples the template store and engagement system (producers) from the index updater and diff summarizer (consumers).
- **Diff Summarization worker**: fires once per (`template_id`, `version`) publish and builds all three disclosure levels in one pass. (1) The raw JSON diff — **Level 3**, technical/support use only. (2) A deterministic structured parser turns each diff path into a labeled old-vs-new record (field, change type, before, after) — **Level 2**, no LLM involved, fully unit-testable. (3) An LLM turns the Level 2 records into a plain-language sentence — **Level 1**, the default view and the only non-deterministic step, which is why it's the only one gated behind human review. Computed once per version and reused by every engagement and firm on that template — the key leverage point.
- **Pending-Update Propagator**: fires on template publish. Queries `EngagementTemplateState` via a GSI on `template_id` for rows behind the new version, and flags them `pending_update=true`. Runs through an SQS buffer so a popular template doesn't block the publish path.
- **Read API** — GET `/firms/{id}/pending-updates`: single query against the index by `firm_id`, filtered on `pending_update`. No engagement loads, no cross-service joins.
- **Update-detail view** — GET `/engagements/{id}/pending-changes`: walks the version hops between `current_version` and `latest_available_version`, returning Level 1 prose by default with Level 2's structured old-vs-new comparison available per hop on expand. Level 3 (raw JSON) is deliberately unreachable from this endpoint — served only through a separate, internal support endpoint, so a non-technical auditor is never one click away from code.

Apply/decline stays entirely inside the existing (out-of-scope) engagement management system; it only needs to fire the same hook on completion so the index reflects the new `current_version`.

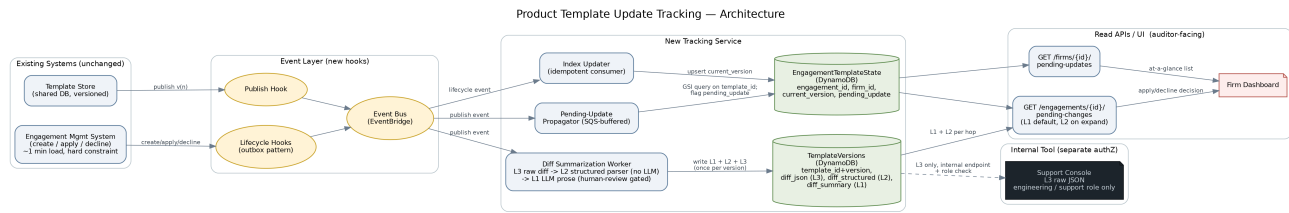


Figure 1 — component and data-flow diagram

2. Implementation Plan

- **Hooks & storage** — add outbox-pattern hooks to engagement create/apply/decline and to template publish; stand up EngagementTemplateState and TemplateVersions; wire the event bus.
- **Index updater** — idempotent consumer that upserts EngagementTemplateState from lifecycle events.
- **Propagator** — fan-out consumer that flags affected engagements on publish, buffered through SQS.
- **Diff summarizer** — deterministic JSON diff → LLM prose summary, gated behind human review before it's customer-visible (at least initially).
- **Read APIs** — firm dashboard query and per-engagement detail view.
- **Backfill** — populate index rows for existing engagements lazily, as they're naturally touched, rather than a forced synchronous migration — avoids paying the 1-minute cost across an entire existing portfolio at once.
- **Rollout** — dark-launch the index (write-only) for one release cycle, validate its current_version against a sample of manually-loaded engagements, before exposing pending_update in the UI.

3. Testing Strategy

- **Unit** — version-comparison logic; idempotent event handling (replaying an event is a no-op); out-of-order events (e.g. a retried “decline” arriving after “apply”); the Level 2 diff-path→structured-record mapping, covered by fixed input/output pairs since it's deterministic and needs no LLM eval.
- **Contract tests** — versioned, backward-compatible schemas for both hook event types.
- **Integration** — publish → verify all affected engagements across firms flip to pending within the propagation SLA; multi-hop accumulation → verify the detail view returns correctly ordered per-version summaries.
- **Load** — fan-out test with a template used by thousands of engagements, confirming SQS buffering holds the propagation SLA without overwhelming downstream tables.
- **Golden-set eval** — a fixed set of template diffs with human-approved reference summaries, scored against new LLM output whenever the prompt or model changes, before summaries are trusted in production.

4. Evaluation & Observability

- **Correctness** — daily reconciliation: load a small random sample of engagements per firm, compare true template_id/version to the index, alert on drift.
- **Latency** — publish-to-fully-propagated time (SLO, e.g. under 15 minutes given weekly cadence); dashboard query latency (sub-second, since no loads are in the read path).
- **Diff quality** — summary edit/rejection rate under the human review gate; periodic sampled audits comparing LLM summaries to raw diffs for material omissions — especially significant given the audit-standards domain.
- **Operational** — DLQ depth on consumers, duplicate/replay rate, LLM failure/timeout rate and how often the system falls back to showing the raw diff.
- **Product signal** — percentage of engagements with pending updates, time-to-decision after a publish — useful beyond engineering health.

5. Failure Modes & Tradeoffs

- **Dropped hook events** → **index drift**. Mitigated with the outbox pattern (event written in the same transaction as the state change) plus daily reconciliation sampling. Worst case is a stale “no pending update” badge, correctable on the next reconciliation pass — not silent loss on the actual apply/decline path, which still reads real engagement state.
- **Level 1 (LLM summary) is wrong or omits something material**. It’s generated from, and checkable against, the deterministic Level 2 record, and gated behind human review before reaching customers — the only one of the three levels that needs that gate, since Levels 2 and 3 are both deterministic. Trust is proportional to how much judgment went into producing each level.
- **Level 3 leaking into the auditor-facing app**. All three levels share the same underlying record, so a routing or permissions bug could expose raw JSON to firm users who can’t act on it. Mitigated by serving Level 3 from a separate, internal-only endpoint and role check — a different code path, not just a hidden UI toggle on the same one.
- **Fan-out cost on widely-used templates**. A template shared across every firm could mean tens of thousands of index rows to update per publish. SQS buffering trades propagation latency for system stability — reasonable at a weekly publish cadence; would need a different design (e.g. streaming/CDC-based propagation) at daily-or-faster cadence.
- **Eventual consistency window**. A user could see “up to date” for a few minutes post-publish before propagation completes. Acceptable since the actual apply/decline decision still reads ground truth, and it’s a strictly better tradeoff than a synchronous 1-minute-per-engagement check.
- **Backfill gap**. There’s no way to know an existing engagement’s true `current_version` without loading it. Lazy backfill means some old, untouched engagements won’t show pending status until next opened — an explicit known gap rather than a forced, costly migration, in keeping with “don’t over-optimize.”